

Command	Description
<code>wsl</code>	Start/open the WSL Ubuntu environment from Windows.
<code>source ~/airflow/venv/bin/activate</code>	Activate the Airflow Python virtual environment.
<code>cd /mnt/d/agent-ai-adk</code>	Navigate to the project directory from WSL.
<code>java -version</code>	Verify Java 17 is available for PySpark.
<code>echo \$JAVA_HOME</code>	Verify the configured Java home path.
<code>echo \$AIRFLOW__CORE__EXECUTION_API_SERVER_URL</code>	Verify Airflow workers use the correct API server URL.
<code>airflow version</code>	Display the installed Airflow version.
<code>airflow dag-processor</code>	Start the Airflow DAG Processor; parses/discovers DAG files.
<code>airflow scheduler</code>	Start the Airflow Scheduler; schedules and executes DAG tasks.
<code>airflow api-server</code>	Start the Airflow API Server on port 8080.
<code>ss -ltnp grep 8080</code>	Verify the Airflow API Server is listening on port 8080.
<code>curl http://127.0.0.1:8080/api/v2/monitor/health</code>	Check Airflow API, scheduler, DAG processor, and metadata DB health.
<code>cd /mnt/d/agent-ai-adk/airflow/dags</code>	Navigate to the DAG directory.
<code>python -m py_compile olist_incremental_pipeline.py</code>	Check the DAG Python file for syntax/indentation errors.
<code>airflow dags list-import-errors</code>	Check whether Airflow has DAG import errors. <code>No data found</code> = no import errors.
<code>airflow dags list</code>	List discovered Airflow DAGs and their paused status.

Command	Description
<code>airflow dags details</code> <code>olist_incremental_pipeline</code>	Display detailed information and health/parsing status of the DAG.
<code>airflow dags unpause</code> <code>olist_incremental_pipeline</code>	Unpause the DAG so it can execute.
<code>airflow dags pause</code> <code>olist_incremental_pipeline</code>	Pause the DAG to prevent execution.
<code>airflow dags trigger</code> <code>olist_incremental_pipeline</code>	Manually trigger a fresh DAG run.
<code>airflow tasks states-for-dag-run</code> <code>olist_incremental_pipeline <RUN_ID></code>	Display the state of every task in a specific DAG run.
<code>python -m spark_pipeline.batch_generator</code>	One-time command to generate the chronological Olist monthly batches.
<code>python -m spark_pipeline.pipeline --</code> <code>batch-id batch_YYYYMM</code>	Manually execute the PySpark pipeline for a specific batch.
<code>ip route show default</code>	Find the Windows host gateway IP from WSL for MySQL connectivity.
<code>which python</code>	Verify which Python executable is currently active.
<code>source ~/.bashrc</code>	Reload persistent environment variables such as Java and Airflow API configuration.
<code>echo \$PYSPARK_PYTHON</code>	Verify which Python executable PySpark uses.
<code>echo \$PYSPARK_DRIVER_PYTHON</code>	Verify which Python executable the PySpark driver uses.

Most important tomorrow

Bash

```
source ~/airflow/venv/bin/activate
cd /mnt/d/agenti-ai-adk/airflow/dags
python -m py_compile olist_incremental_pipeline.py
airflow dags list-import-errors
airflow dags details olist_incremental_pipeline
```

